MONVEX

AI-POWERED PERSONAL FINANCIAL INTELLIGENCE SYSTEM

A Project Report submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF SCIENCE

(COMPUTER SCIENCE) — SEMESTER V

University Of Mumbai

DEPARTMENT OF COMPUTER SCIENCE



Chembur Trombay Education Society's N.G. Acharya & D.K. Marathe College of Arts, Science & Commerce

Shri N.G. Acharya Marg, Chembur, Mumbai – 71

Submitted by

ANSARI DANISH ASLAM

Class: TYCS A Roll No.: 6

Student ID: 5072732

Seat No.: [STUDENT_SEAT_NUMBER] PRN: [STUDENT_PRN_NUMBER]

Under the guidance of

Prof. [GUIDE_FULL_NAME] — [GUIDE_DESIGNATION]

ACADEMIC YEAR: 2026–2027

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

Particular	Details
Student name	ANSARI DANISH ASLAM
Class / division	TYCS A
College roll number	6
Student ID	5072732
University seat number	[STUDENT_SEAT_NUMBER]
PRN / PNR	[STUDENT_PRN_NUMBER]
Academic year	2026–2027
Project title	MONVEX: AI-Powered Personal Financial Intelligence System
Project area	Web engineering, financial data processing, machine learning and generative AI
Project guide	Prof. [GUIDE_FULL_NAME]
Guide designation / qualification	[GUIDE_DESIGNATION] / [GUIDE_QUALIFICATIONS]
Teaching experience	[GUIDE_YEARS_EXPERIENCE] years
Decision	[APPROVED / NOT APPROVED — TO BE COMPLETED BY THE INSTITUTION]
Approval date	[APPROVAL_DATE]

Guide signature: _____ Student signature: _____

This proforma is supplied for completion by the college. No approval, signature, stamp, or examination result is asserted in this report.

CERTIFICATE

This is to certify that the project entitled “MONVEX: AI-Powered Personal Financial Intelligence System” is the work of ANSARI DANISH ASLAM (Class TYCS A, College Roll No. 6, Student ID 5072732; University Seat No. [STUDENT_SEAT_NUMBER], PRN [STUDENT_PRN_NUMBER]) and is submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science, Semester VI, at Chembur Trombay Education Society’s N.G. Acharya & D.K. Marathe College of Arts, Science & Commerce, during academic year 2026–2027. The university affiliation and certificate are subject to institutional verification and approval.

Internal Guide: _____ Head of Department: _____

Prof. [GUIDE_FULL_NAME] Prof. [HOD_FULL_NAME]

External Examiner: _____ Principal: _____

[EXTERNAL_EXAMINER_NAME] Dr. [PRINCIPAL_FULL_NAME]

Date: [CERTIFICATION_DATE] College seal: [TO BE AFFIXED BY THE INSTITUTION]

DECLARATION

I, ANSARI DANISH ASLAM (Class TYCS A, College Roll No. 6, Student ID 5072732), declare that this report describes the MONVEX project work undertaken for the B.Sc. Computer Science Semester VI programme at Chembur Trombay Education Society's N.G. Acharya & D.K. Marathe College of Arts, Science & Commerce, under the guidance of Prof. [GUIDE_FULL_NAME]. The implementation claims in this report are limited to the active repository and the evidence identified in the report. External sources are acknowledged in Chapter 8. This declaration must be reviewed and signed by the student before submission.

Place: Shri N.G. Acharya Marg, Chembur, Mumbai – 71 Date: [SUBMISSION_DATE]

Student signature: _____ ANSARI DANISH ASLAM

ACKNOWLEDGEMENT

I express my gratitude to Chembur Trombay Education Society's N.G. Acharya & D.K. Marathe College of Arts, Science & Commerce, the Department of Computer Science, and my project guide, Prof. [GUIDE_FULL_NAME], for the academic support and feedback provided during this project. I also acknowledge the maintainers of the frameworks, libraries, standards, and technical documentation cited in this report. Guide acknowledgement details remain subject to confirmation.

ABSTRACT

MONVEX is a personal finance application that combines transaction recording with budgeting, savings goals, portfolio summaries, forecasting, receipt processing, and AI-assisted explanation. The active repository contains a Next.js and React web client, a Django REST API, a relational data model, and a Tauri-based Windows desktop wrapper. The backend can use PostgreSQL through DATABASE_URL and has a SQLite development fallback; the Render and Neon deployment settings are configuration evidence, not proof of a currently running cloud service.

Users can record income, expenses, and transfers manually or through natural-language parsing. An optional TF-IDF and Multinomial Naive Bayes pipeline suggests a transaction category. Uploaded receipts pass file-signature checks and can be processed through a configured Gemini model; extracted values are staged for user review before being linked to a ledger record. Budget and goal views summarize progress, while financial-health and cash-flow calculations provide rule-based estimates. A conversational assistant can use scoped financial context and an external generative model when credentials and provider access are configured. The system does not connect to bank accounts through Open Banking providers.

The reviewed source demonstrates a multi-user web application with per-user query scoping, JWT authentication, optional TOTP multi-factor authentication, fixed-point decimal model fields, API throttling, and upload validation. Several dossier-reported checks and screenshot audit results are available, but their full original execution logs were not present for independent reproduction during this report build. The report therefore distinguishes repository verification from test results reported by Phase 1. Student and college details visible on the supplied identity card have been entered; university, seat/PRN, guide, approval, and signature details remain for institutional completion.

TABLE OF CONTENTS

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL 2



..... 3

CERTIFICATE 3

DECLARATION 4

ACKNOWLEDGEMENT 5

ABSTRACT 6

TABLE OF CONTENTS 7

LIST OF FIGURES 11

LIST OF TABLES 12

ABBREVIATIONS 13

1. INTRODUCTION 15

1.1 Background 15

1.2 Objectives 15

1.3 Purpose, Scope and Applicability 15

1.3.1 Purpose 15

1.3.2 Scope 16

1.3.3 Applicability 16

1.4 Achievement 16

1.5 Organization of Reports 16

2. SURVEY OF TECHNOLOGIES 19

2.1 Frontend: Next.js, React and TypeScript 19

2.2 Styling, UI state and data fetching 20

2.3 Backend: Django and Django REST Framework 20
2.4 PostgreSQL and relational persistence 20
2.5 Machine learning and numerical libraries 20
2.6 Generative AI and image processing 21
2.7 Desktop, report generation and test tools 21
2.8 Security standards and relevant references 21
3. REQUIREMENTS AND ANALYSIS 23
3.1 Problem Definition 23
3.2 Requirement Specification 23
3.2.1 Non-functional requirements 23
3.3 Planning and Scheduling 24
3.4 Hardware and Software Requirements 24
3.5 Preliminary Product Description 24
3.6 Conceptual Models 25
3.6.1 Data Flow Diagram 26
3.6.2 Entity–Relationship Diagram 26
3.6.3 Class Diagram 27
3.6.4 Use Case Diagram 27
3.6.5 Sequence Diagram 28
3.6.6 System Flowchart 29
4. SYSTEM DESIGN 31
4.1 Basic Modules in Detail 31
4.1.1 Web client and navigation 31
4.1.2 Authentication and profile 31
4.1.3 Transaction ledger and ingestion 31

4.1.4 Budgets and savings goals 32
4.1.5 Portfolio, debt and forecast 32
4.1.6 Analytics and AI 32
4.1.7 Receipt processing and reporting 32
4.1.8 Desktop wrapper and external services 33
4.1.9 System architecture 33
4.1.10 API groups and integrations 33
4.2 Data Design 34
4.2.1 Data Integrity and Constraints 34
4.3 GUI Screenshots 35
5. IMPLEMENTATION AND TESTING 42
5.1 Implementation Approaches 42
5.1.1 Coding Details and Code Efficiency 42
5.1.1.1 Code Details 42
5.1.1.2 Code Efficiency 43
5.2 Testing Approach 43
5.2.1 Unit Testing 43
5.2.2 Integrated Testing 44
6. SYSTEM DESIGN 46
6.1 Test Reports 46
6.1.1 Representative test cases 46
7. CONCLUSIONS 49
7.1 Conclusion 49
7.1.1 Significance of the System (Advantages) 49
7.1.2 Limitations of the System (Disadvantages) 49

7.1.3 Future Scope of the Project 50

8. REFERENCES 52

LIST OF FIGURES

Figure 1 — Data Flow Diagram, Level 0 (context boundary) 26
Figure 2 — Data Flow Diagram, Level 1 (selected processes) 26
Figure 3 — ERD of selected core entities; User uses an integer PK while most domain records use UUIDs 27
Figure 4 — Class and service relationships (selected implementation) 27
Figure 5 — Use case view of current web/API scope 28
Figure 6 — Sequence of staged receipt extraction and user confirmation 28
Figure 7 — Flowchart for transaction entry and review 29
Figure 8 — MONVEX logical architecture and data movement across modules 33
Figure 9 — Public landing page (registered UI capture UI-01; local audit asset) 36
Figure 10 — Login (registered UI capture UI-02; local audit asset) 36
Figure 11 — Dashboard (registered UI capture UI-03; local audit asset) 37
Figure 12 — Transaction ledger (registered UI capture UI-05; local audit asset) 37
Figure 13 — AI copilot workspace (registered UI capture UI-07; local audit asset) 38
Figure 14 — Budget overview (registered UI capture UI-09; local audit asset) 38
Figure 15 — Savings goals (registered UI capture UI-10; local audit asset) 39
Figure 16 — Receipt review workspace (registered UI capture UI-15; local audit asset) 39
Figure 17 — Analytics view (registered UI capture UI-11; local audit asset) 40

LIST OF TABLES

Table 1 — MONVEX technology roles and version evidence 19
Table 2 — Functional requirements derived from the dossier and active source 23
Table 3 — Non-functional requirements and evidence limits 24
Table 4 — Proposed project schedule; illustrative relative weeks, not actual dates 24
Table 5 — Hardware and software requirements 24
Table 6 — API groups and their integration boundaries; endpoint examples follow active URL routing 34
Table 7 — Selected database entities, keys, and data roles 34
Table 8 — Reported test and verification totals, with evidence qualification 46
Table 9 — Representative unit, integration, browser, and viewport cases 47

ABBREVIATIONS

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
DRF	Django REST Framework
E2E	End-to-End
ERD	Entity–Relationship Diagram
JWT	JSON Web Token
MFA	Multi-Factor Authentication
ML	Machine Learning
NLP	Natural Language Processing
OCR	Optical Character Recognition
ORM	Object-Relational Mapper
PK / FK	Primary Key / Foreign Key
TOTP	Time-Based One-Time Password

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

1.1 Background

Personal financial information is often scattered among bank statements, receipts, budget sheets, and separate investment or debt records. Manual review makes it difficult to form a consistent picture of cash flow and spending patterns. MONVEX addresses this organizational problem by providing a single interface for a user-entered financial ledger and derived summaries. Its design combines familiar record keeping with optional automation: text parsing, category suggestions, receipt extraction, and conversational summaries.

MONVEX should be understood as a personal tracking and decision-support system. It does not act as a bank, hold customer funds, or retrieve live bank transactions. The quality of its summaries depends on the completeness and accuracy of records entered or reviewed by the user. Its projections and AI outputs are informational and do not replace professional financial or tax advice.

1.2 Objectives

- Provide authenticated users with a ledger for income, expenses, and transfers.
- Organize transactions into categories and support search, filters, and export.
- Help users set category budgets and savings goals and review progress.
- Summarize assets and liabilities and calculate net worth from user-maintained values.
- Offer rule-based financial indicators, cash-flow projections, and debt payoff comparisons.
- Reduce repetitive entry through natural-language parsing, ML category suggestions, and reviewable receipt OCR.
- Provide privacy-conscious multi-user access controls and make the limits of external AI and deployment explicit.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose is to demonstrate an integrated software system that accepts personal finance data, validates and stores it, and presents understandable summaries. The academic focus is the

interaction between frontend state, authenticated APIs, relational data, deterministic calculations, and selected ML/AI integrations.

1.3.2 Scope

The active scope includes the responsive web application, Django API, database models, and a Tauri Windows shell. Function groups cover identity, transaction management, budgets, goals, net worth, debt planning, forecasts, analytics, receipt review, reports, notifications, search, and AI assistance. A Flutter app is described in accessible Git history but is absent from the current checkout and is not part of the active runnable release documented here. Open Banking synchronization is future scope. Render/Neon files describe a deployment target; no live deployment was independently confirmed.

1.3.3 Applicability

The system is intended for individuals who want to maintain a private personal ledger and review their own manually supplied financial information. It can support students learning about full-stack architecture, API design, data modelling, and applied analytics. It is not a regulated financial institution, certified accounting system, or substitute for professional advice.

1.4 Achievement

Source inspection confirms the core application structure and major feature flows. The repository includes a Next.js App Router frontend, Django apps and services, fixed-point monetary fields, a category-classification pipeline, receipt validation and OCR services, a financial assistant integration, export functionality, and a Windows desktop wrapper. Test and responsive audit outcomes are reported in the dossier and in an audit-results JSON file, but those outcomes are not equivalent to a fresh run of each backend and browser suite for this report.

1.5 Organization of Reports

Chapter 1 introduces the problem and scope. Chapter 2 surveys the technologies. Chapter 3 specifies requirements, constraints, and conceptual models. Chapter 4 describes architecture, modules, data design, and verified UI captures. Chapter 5 discusses implementation and testing methods. Chapter 6 preserves the prescribed template title “SYSTEM DESIGN” and contains the

test reports under section 6.1. Chapter 7 concludes with significance, limitations, and future scope using the supplied hierarchy. Chapter 8 lists the technical references.

CHAPTER 2

SURVEY OF TECHNOLOGIES

2. SURVEY OF TECHNOLOGIES

MONVEX uses a layered web architecture. The browser client handles navigation, forms, state, and visualizations; the backend validates requests and applies domain rules; database models preserve records; and selected external services provide identity or generative inference. The version information below is stated at the precision supported by repository manifests and lockfiles.

Technology	Evidence-supported version / status	Role in MONVEX
Next.js / React	Next.js 14.2.35; React 18.3.1 in package lock	App Router web interface, server rendering/build, reusable views.
TypeScript	Manifest range 5.5; lockfile exact version 5.9.3	Typed component and API contracts.
Tailwind CSS	Manifest range 3.4; lockfile 3.4.19	Utility styling and responsive visual system.
TanStack Query	Manifest range 5.51; lockfile 5.102.2	Asynchronous server data cache and query lifecycle.
Django / DRF	requirements allow Django >=5,<6; DRF >=3.15	ORM, request routing, serializers, permissions, API responses.
Database	PostgreSQL URL path configured; SQLite fallback in settings	Relational persistence; active production engine not confirmed.
Scikit-learn / NumPy	Requirements ranges, no Python lockfile	TF-IDF feature extraction and Naive Bayes category suggestions.
Google Gemini	Model defaults to gemini-2.0-flash; API key configuration required	Optional conversational responses and receipt extraction.
Tauri / Rust	Tauri 1.x configured in Cargo manifest	Windows desktop wrapper around the web client.
ReportLab	Dependency present in requirements	Backend PDF statement generation.
Playwright	@playwright/test declared in web package	Browser navigation and responsive audit scripts.
Render / Neon	Blueprint and environment configuration present	Intended cloud deployment configuration; runtime not verified.

Table 1 — MONVEX technology roles and version evidence

2.1 Frontend: Next.js, React and TypeScript

Next.js provides the web application framework and App Router structure, while React components compose pages and shared layouts. This organization supports route-specific screens alongside shared navigation, authentication, and theme state. TypeScript describes data structures passed between components and API clients, reducing mismatches that can be caught during compilation. The lockfile resolves newer compatible patch and minor versions than the original dossier's figures; this report uses the checked-in lockfile where it records an exact frontend version and avoids presenting manifest ranges as exact versions [1–3].

The trade-off is operational complexity compared with a static multipage site: a Node runtime and build pipeline are needed for the configured standalone application, and server data remains dependent on API availability. No performance benchmark is claimed here.

2.2 Styling, UI state and data fetching

Tailwind CSS supplies utility classes and responsive styling conventions. TanStack Query manages server-state requests, caching, and invalidation; React contexts manage application-wide concerns such as authentication, theme, and navigation state. The project also declares Zustand for selected client state. These tools separate remote API state from transient presentation state, but correct cache invalidation and clear empty/error states remain important responsibilities of the application [4–6].

2.3 Backend: Django and Django REST Framework

Django provides routing, model definitions, migrations, settings, and authentication primitives. Django REST Framework adds API views, serializers, authentication/permission checks, pagination, and throttling. The dependency file specifies version ranges rather than a resolved Python lockfile; therefore the report identifies the Django 5.x family and the DRF minimum range instead of repeating unverified patch versions [7–9].

The backend separates endpoint orchestration from services for finance calculations, forecasting, OCR, parsing, validation, and PDF creation. This supports reuse, although modules still require integration tests to confirm interactions under real database and provider conditions.

2.4 PostgreSQL and relational persistence

The data model is relational: users own transactions and financial records, categories classify transactions, and related objects are joined through foreign keys. PostgreSQL is supported through DATABASE_URL and a Neon-oriented Render blueprint; local settings also include a SQLite fallback. As no live database connection or deployed service was inspected, PostgreSQL 16 is treated as a configuration target rather than a verified active engine. Relational constraints and indexes support data integrity and common user/date queries [10–11].

2.5 Machine learning and numerical libraries

The categorizer combines TF-IDF text features with a Multinomial Naive Bayes classifier. It uses merchant names and descriptions to suggest a category and confidence value when a category is absent. The output is a suggestion derived from the project's training examples, not a universal or independently benchmarked classification accuracy claim. Scikit-

learn supplies the vectorizer/classifier pipeline; NumPy supports numerical operations in the ecosystem. No accuracy percentage is asserted [14–15].

2.6 Generative AI and image processing

The Gemini client reads configuration for a model defaulting to gemini-2.0-flash and requires an API key for external calls. Receipt OCR validates the uploaded content type/signature, requests structured extraction, and stages the resulting values for user review. The copilot can build a limited financial context from application data and then request a generated response. External availability, current provider model access, latency, and output accuracy were not measured; users should review generated results and avoid treating them as professional advice [13].

2.7 Desktop, report generation and test tools

Tauri wraps the web interface for Windows using a Rust native shell and WebView2. Cargo configuration proves a Tauri 1.x dependency, but runtime behavior is not independently certified in this report. ReportLab is used by backend services to create monthly PDF statements. Playwright scripts and audit artifacts support browser navigation and viewport checks. Each tool has a limited role: an installed/configured library does not by itself prove a successful deployment or test execution [16–17].

2.8 Security standards and relevant references

JWT and TOTP implementations are related to published IETF standards. The system also contains web security controls that align with common application security guidance, such as scoped data access, request throttling, secure cookie settings, and content validation. These controls reduce specific risks but do not establish an absolute security guarantee [18–20].

CHAPTER 3

REQUIREMENTS AND ANALYSIS

3. REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

A person tracking finances across receipts, statements, and separate worksheets must repeatedly enter and reconcile information. This can make monthly spending patterns, budget variance, and estimated net worth harder to review. The proposed system provides one authenticated workspace in which users enter or import selected information, classify transactions, and inspect computed summaries.

The design intentionally does not solve automatic bank aggregation. Users provide information through manual entry, natural-language parsing, receipt upload, or supported import flows. All calculations depend on the records and assumptions available in the application.

3.2 Requirement Specification

ID	Functional requirement	Acceptance intent / evidence status
FR-01	Register and authenticate a user	Create identity and profile; reject invalid credentials. Source and dossier-reported test evidence.
FR-02	Protect user-specific records	Scope reads/writes to authenticated user; cross-user test is dossier-reported.
FR-03	Record and maintain ledger entries	Create, retrieve, edit, delete and filter income/expense/transfer records.
FR-04	Categorize transactions	Accept explicit category or use optional ML suggestion; user can review.
FR-05	Parse short natural-language entries	Extract supported amount/date/text fields; incomplete parses require correction.
FR-06	Manage budgets and savings goals	Calculate budget consumption and record goal contributions.
FR-07	Review assets, liabilities and net worth	Summarize user-maintained balances; values are not bank-verified.
FR-08	Provide financial summaries	Expose dashboard, health-score and forecast calculations with their assumptions.
FR-09	Process receipt images	Check file signature, request optional OCR, stage results for confirmation.
FR-10	Offer AI assistance	Use scoped context and configured provider; handle missing provider/configuration.
FR-11	Export financial reports	Provide CSV and monthly PDF outputs for selected user data.
FR-12	Support desktop access	Wrap the web client using the configured Tauri Windows shell.

Table 2 — Functional requirements derived from the dossier and active source

3.2.1 Non-functional requirements

Quality area	Requirement / design evidence	Limitation
Privacy	User-specific foreign keys and request-user queryset scoping.	Must be verified for each endpoint; identifiers alone are not authorization.
Financial integrity	Decimal fields for monetary values and model/database constraints.	Input validation and domain rules must also be checked.
Security	JWT lifecycle, optional TOTP, throttling, file-signature checks, origin settings.	MFA secret field is plain CharField in active model; storage encryption not evidenced.
Usability	Responsive Next.js views, clear empty/loading states and exports.	Registered screenshot audit has missing paths; captures show empty/demo states.
Maintainability	Separate frontend, API, domain services, models, and tests.	No quantitative maintainability or runtime benchmark measured.
Availability	Health/readiness endpoints and deployment blueprint.	No live cloud availability or uptime verified.
Interoperability	JSON APIs, CSV and PDF output.	No live Open Banking connector in active source.

Table 3 — Non-functional requirements and evidence limits

3.3 Planning and Scheduling

No authoritative dated project plan was independently established. The dossier's Gantt fragment marks several activities complete while using dates extending beyond the dossier's stated inspection date. To avoid misrepresenting project history, Table 3.3 is a proposed planning illustration only; it is not a record of actual completion dates.

Illustrative phase	Indicative duration	Key work products	Status
1. Requirements and evidence review	Week 1	Scope, user needs, acceptance criteria	Planning illustration
2. Architecture and data design	Weeks 2–3	Component boundaries, ERD, API design	Planning illustration
3. Core application development	Weeks 4–7	Identity, ledger, budgets, goals, dashboard	Planning illustration
4. Analytics and integrations	Weeks 8–9	Forecasting, ML suggestions, OCR/AI paths	Planning illustration
5. Desktop and interface refinement	Week 10	Tauri shell, responsive states	Planning illustration
6. Verification and documentation	Weeks 11–12	Tests, screenshots, final report and review	Planning illustration

Table 4 — Proposed project schedule; illustrative relative weeks, not actual dates

3.4 Hardware and Software Requirements

Category	Requirement / target	Evidence qualification
Client device	Desktop/laptop or modern smartphone with a current browser.	Responsive captures recorded at 1440×900 and 390×844 in the dossier.
Development runtime	Node.js for Next.js; Python for Django; Rust toolchain for Tauri builds.	Package and build configuration present; no single mandatory machine specification found.
Database	PostgreSQL connection for deployed configuration; SQLite fallback in Django settings.	Configuration only; live service not confirmed.
Network	Connectivity to API and optional Google identity/AI providers.	Required for remote API/provider use; offline provider behavior not established.
Build and test tools	npm, Django test tooling, Playwright, and optional WebView2 tooling.	Declared scripts and evidence artifacts; fresh full suite not run during this report build.

Table 5 — Hardware and software requirements

3.5 Preliminary Product Description

MONVEX is a client/server personal finance application. A user signs in through the web client, submits a validated API request, and receives a response derived from user-scoped records. Transaction ingestion can be direct or assisted. In the receipt flow, validation occurs before OCR, the candidate record remains reviewable, and only confirmation creates the associated ledger transaction. Budget, goal, analytics, and portfolio screens read their own records or derived aggregates. CSV/PDF outputs provide portable views of selected data.

Feature boundaries are material: Open Banking sync is planned, external AI requires valid credentials and network access, and the historical Flutter code is not present in the current checkout. Render/Neon configuration is not the same as an active production deployment.

3.6 Conceptual Models

The following models are explanatory views of the current web/API design. They show selected entities and flows rather than every endpoint, table, or runtime state. Figure 3.1 and 3.2 split the required DFD into context and functional decomposition levels.

3.6.1 Data Flow Diagram



Figure 1 — Data Flow Diagram, Level 0 (context boundary)

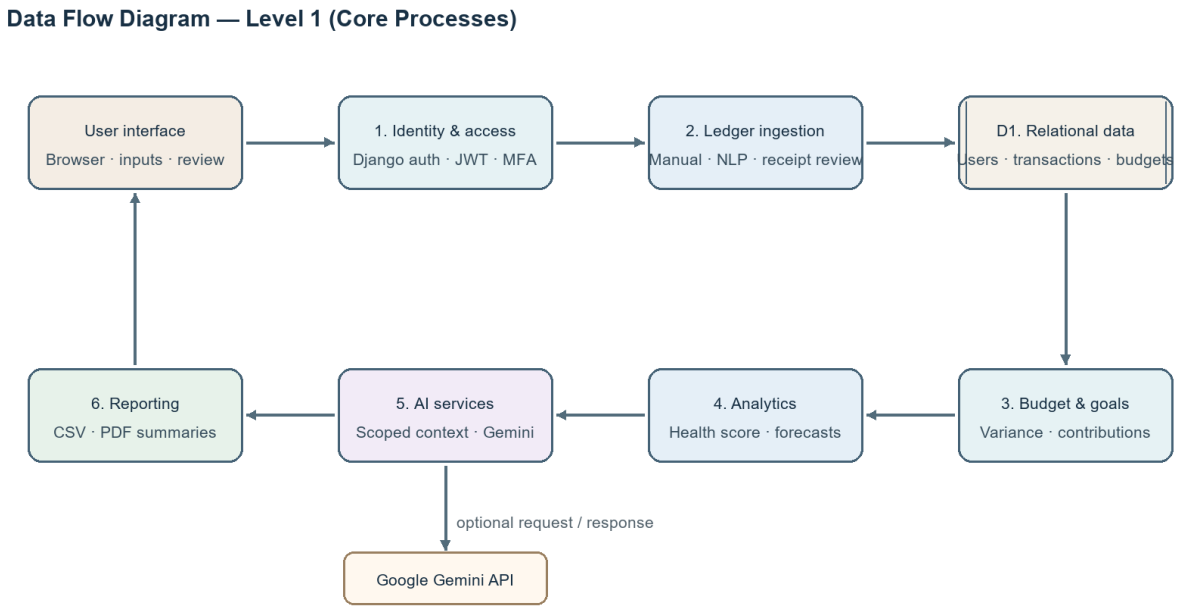


Figure 2 — Data Flow Diagram, Level 1 (selected processes)

3.6.2 Entity–Relationship Diagram

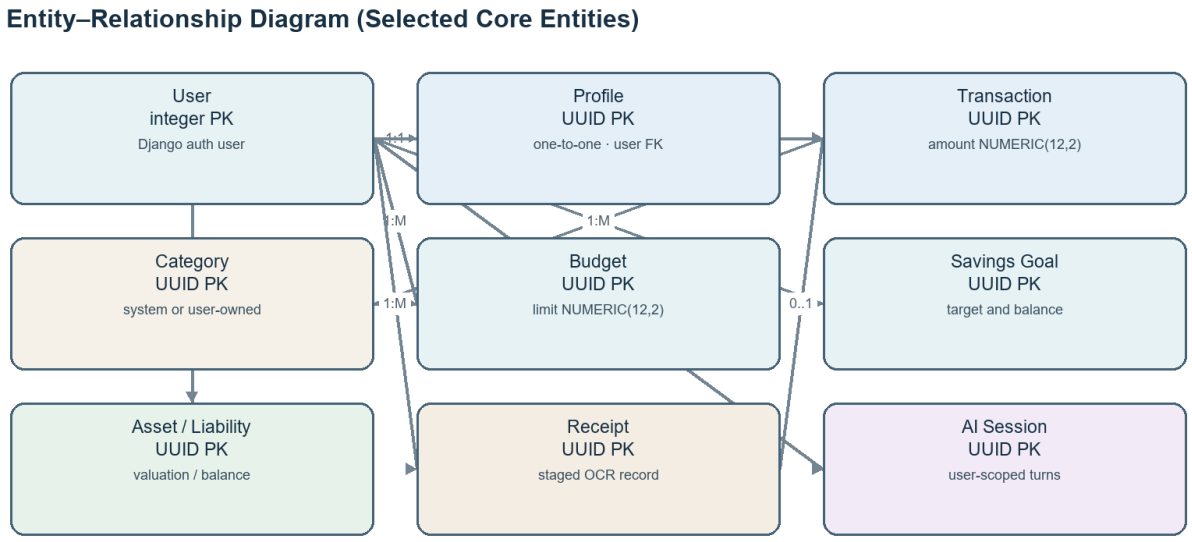


Figure 3 — ERD of selected core entities; User uses an integer PK while most domain records use UUIDs

3.6.3 Class Diagram

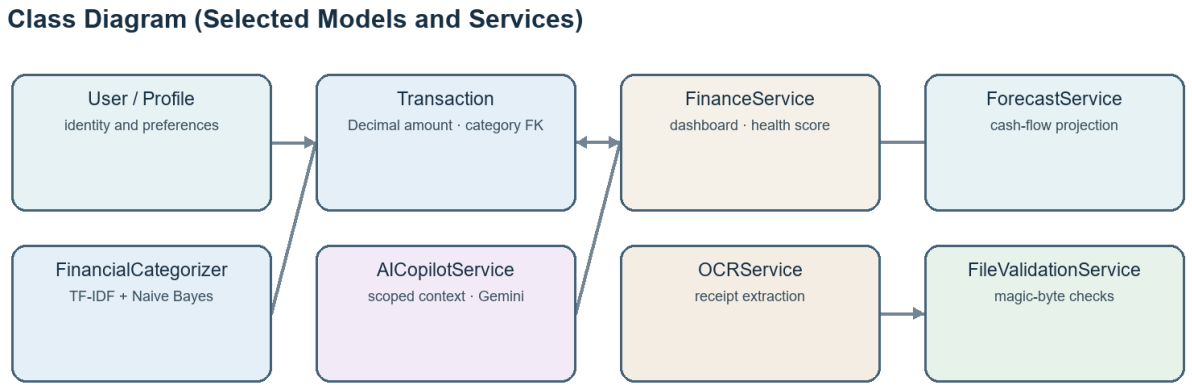


Figure 4 — Class and service relationships (selected implementation)

3.6.4 Use Case Diagram

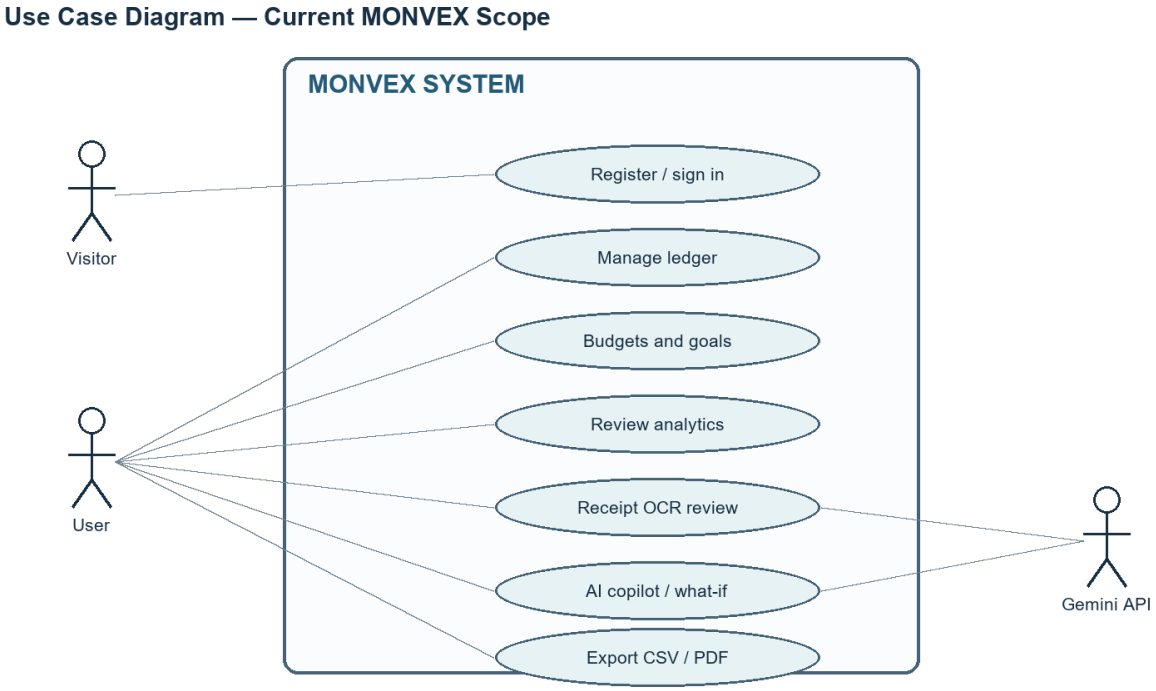


Figure 5 — Use case view of current web/API scope

3.6.5 Sequence Diagram

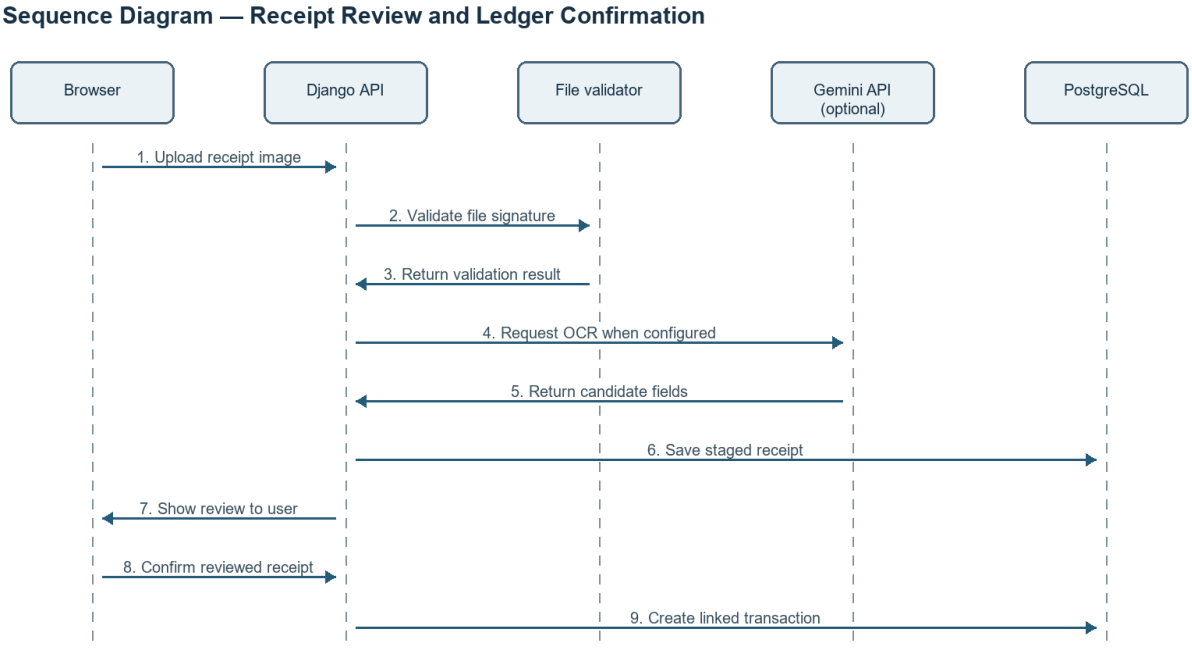


Figure 6 — Sequence of staged receipt extraction and user confirmation

3.6.6 System Flowchart

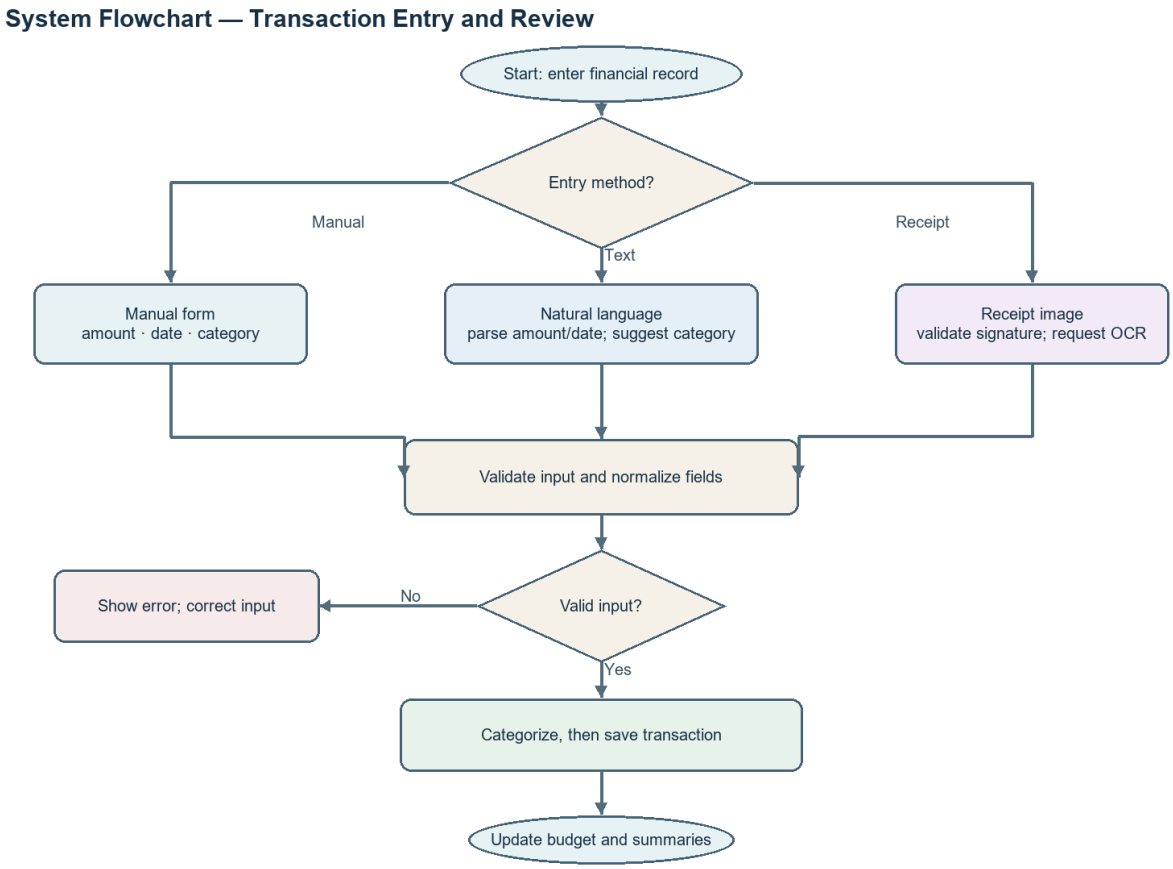


Figure 7 — Flowchart for transaction entry and review

CHAPTER 4

SYSTEM DESIGN

4. SYSTEM DESIGN

4.1 Basic Modules in Detail

4.1.1 Web client and navigation

The Next.js App Router organizes public and protected route pages, shared shell components, and route-level financial workspaces. The app shell holds navigation and global states such as theme and authentication. Page components call typed API modules and query hooks. Loading, empty, and error states matter because an API response may be delayed, unavailable, or contain no records. The inspected captures mostly show empty account states with zero summaries; they are genuine UI views but do not prove provider-backed account data.

4.1.2 Authentication and profile

Registration and login views send credentials to Django endpoints. The backend uses Django's user model and a one-to-one Profile extension. JWT access/refresh handling is configured with refresh-cookie behavior and token rotation settings; TOTP setup and verification endpoints exist. Google identity support validates ID tokens through a service. These are code-level controls; actual provider configuration and live email delivery are environment-dependent. The active Profile model stores mfa_secret as a normal character field, and no encryption wrapper was found in the inspected model/view path, so this report does not claim encrypted-at-rest TOTP secrets.

4.1.3 Transaction ledger and ingestion

The ledger supports income, expense, and transfer types and stores a positive Decimal amount, date, optional category and merchant, source, confidence, and text. The code uses a single transaction record per entry; it is not a general double-entry accounting ledger with balancing journal lines. A service normalizes known merchant spellings and may ask the classifier for a category when none is supplied. The user can add, filter, edit, delete, and export entries through authenticated API routes.

4.1.4 Budgets and savings goals

Budget records connect a user, category, limit, and period. The unique user/category/period constraint prevents duplicate budget rows for that combination. Budget views compare recorded spending with a limit and present consumed/remaining amounts. Goal records track a target, current amount, optional deadline, and status; contributions are logged as related rows. Calculated progress is only as accurate as the supplied transaction and contribution data.

4.1.5 Portfolio, debt and forecast

Asset and Liability records store user-entered values and debt parameters. Net worth is derived from asset totals less liability balances; the system does not independently verify market prices or account balances. Debt planning compares avalanche and snowball ordering using stored interest and payment assumptions. The forecast service projects cash flow from historical spending and recurring commitments; it is a simplified estimate rather than a bank-grade forecast or guaranteed prediction.

4.1.6 Analytics and AI

Analytics endpoints aggregate transaction history and profile values into dashboard metrics and a 0–100 health score. This is an application-defined index, not a clinical or regulated rating. The copilot assembles scoped context and may call Gemini; a deterministic fallback is present in source. AI messages can be stored in conversation models. Prompt boundaries and disclaimers reduce misuse risk but cannot guarantee factual correctness, confidentiality outside the application boundary, or suitability of advice.

4.1.7 Receipt processing and reporting

Receipt uploads are checked for allowed file signatures, parsed by a configured OCR service, and stored as reviewable Receipt objects. A user may reject the staged item or confirm it, which links it to a transaction. The service also exposes secure image retrieval. Monthly PDF generation is implemented with ReportLab; CSV export uses a data stream. Provider access, successful OCR for arbitrary images, and deployment storage persistence are not established merely by service code.

4.1.8 Desktop wrapper and external services

The Tauri configuration and Rust source wrap the web experience as a Windows desktop application. It remains a web-backed client and does not create a distinct financial data store. Google Gemini, Google OAuth, and SMTP are optional integrations controlled by environment configuration. Open Banking providers such as Plaid or Yodlee are not implemented.

4.1.9 System architecture

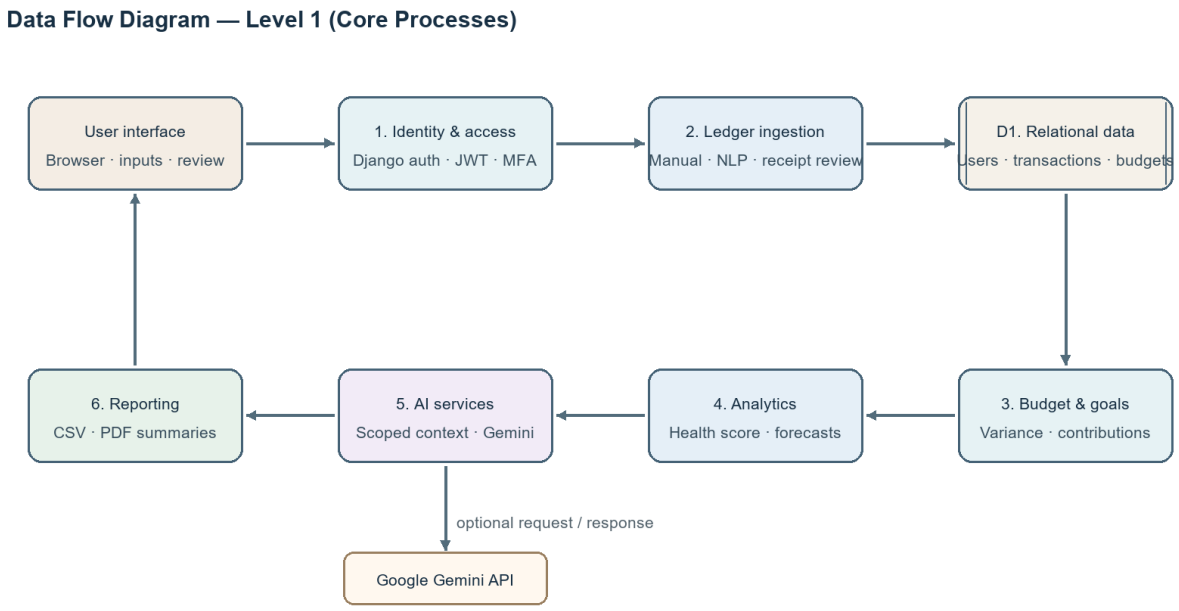


Figure 8 — MONVEX logical architecture and data movement across modules

The browser communicates with versioned JSON endpoints under the API route tree. Django views apply authentication, permissions, serializer validation, and domain-service calls. The ORM maps domain records to the selected relational database. External provider calls are isolated in service clients and depend on runtime configuration. Health and readiness endpoints support deployment monitoring but do not establish live availability.

4.1.10 API groups and integrations

API group	Representative operations	Access / integration boundary
Authentication	Register, login, logout, refresh, profile, MFA, Google identity	Registration/login are public; profile and MFA management require an authenticated session.
Transactions	Ledger CRUD, categories, recurring records, natural-language parse, duplicates	Financial records are user-scoped; external bank sync is not present.
Budgets and goals	Create/list limits; budget overview; goal contributions	Authenticated data endpoints; validations apply to monetary inputs and ownership.
Analytics and portfolio	Dashboard, health score, forecast, assets, liabilities, net worth, debt planning	Authenticated summaries derived from stored records and user-maintained balances.

API group	Representative operations	Access / integration boundary
Receipts and AI	Receipt upload/review/confirm; copilot chat; what-if; insights	Gemini calls require provider configuration; receipt extraction is staged for user review.
Reports and utility	CSV export, monthly JSON/PDF report, search, notifications, health probes	Report/search endpoints use account context where financial data is returned; probes are operational endpoints.

Table 6 — API groups and their integration boundaries; endpoint examples follow active URL routing

4.2 Data Design

The project uses Django's default auth.User as the identity root. Its primary key is integer-valued. Profile and most domain entities define UUID primary keys. Foreign keys use the target key type, so `user_id` is an integer while category, transaction, and other domain keys are commonly UUIDs. This is a mixed-key design; the dossier's generalization that every model uses UUIDs is incorrect.

Entity	Primary key	Selected fields and relationships	Purpose
auth.User	Integer	username, email, password hash; one-to-one Profile	Identity and credential root.
Profile	UUID	user FK unique; currency, monthly_income, settings, mfa_secret	User preferences and finance baseline.
Category	UUID	nullable user FK; name/type; system-default flag	Income/expense classification.
Transaction	UUID	user FK integer; optional category/merchant UUID FKs; amount NUMERIC(12,2)	Single-entry cash-flow record.
Budget	UUID	user FK; category FK; limit NUMERIC(12,2); period; unique combination	Category spending limit.
SavingsGoal	UUID	user FK; target/current NUMERIC(12,2); contributions	Goal progress.
Asset / Liability	UUID	user FK; NUMERIC(14,2) valuations and balances	User-maintained balance sheet.
Receipt	UUID	user FK; total NUMERIC(12,2); status; JSON items; optional transaction FK	Staged OCR record and confirmation link.
ConversationSession / Message	UUID	session user FK; messages FK session; JSON data	AI conversation history.
Notification / AnomalyEvent	UUID	user FK; optional transaction FK; status and metadata	Alerts and review state.

Table 7 — Selected database entities, keys, and data roles

4.2.1 Data Integrity and Constraints

- Monetary attributes use Django DecimalField/SQL NUMERIC precision; calculations should preserve Decimal values and avoid binary floating-point conversion. This does not prove every path is free of rounding mistakes.
- Foreign-key behavior varies by relation: transaction categories and merchants may become null when removed; user-owned records generally cascade with account deletion.
- Budget uniqueness is enforced across user, category, and period. GoogleIdentity adds uniqueness for provider and provider subject.
- User scoping is implemented through authenticated request context in relevant API querysets. UUIDs make enumeration less predictable but do not replace authorization.

- Receipt file signatures are checked before external processing; file size, storage durability, and provider-specific error handling remain separate concerns.
- The active MFA secret field is not visibly encrypted in the reviewed model path. This is a security limitation and a candidate for remediation before sensitive production use.

4.3 GUI Screenshots

The UI screenshot register lists 33 captures. Paths for screenshots under the `artifacts/` directory were not present in the current checkout; verified report captures were selected from existing audit_screenshots assets. The selected files were opened in a contact sheet and visually checked for correct MONVEX pages, broken layout, obvious credentials, and personal financial details. They show mostly empty/demo states with zero-value summaries; no genuine user account data is intentionally reproduced. The dossier's synthetic-data provenance is recorded, but the screenshots themselves do not independently prove a live service session.

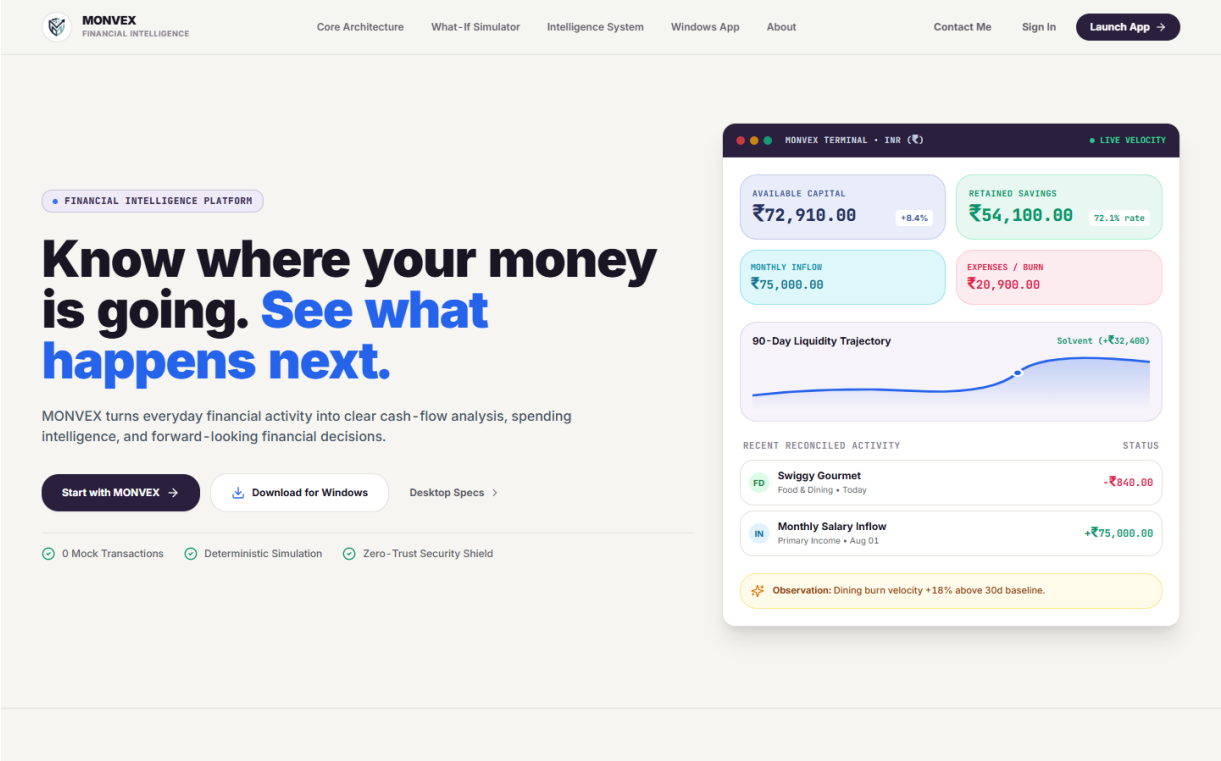


Figure 9 — Public landing page (registered UI capture UI-01; local audit asset)

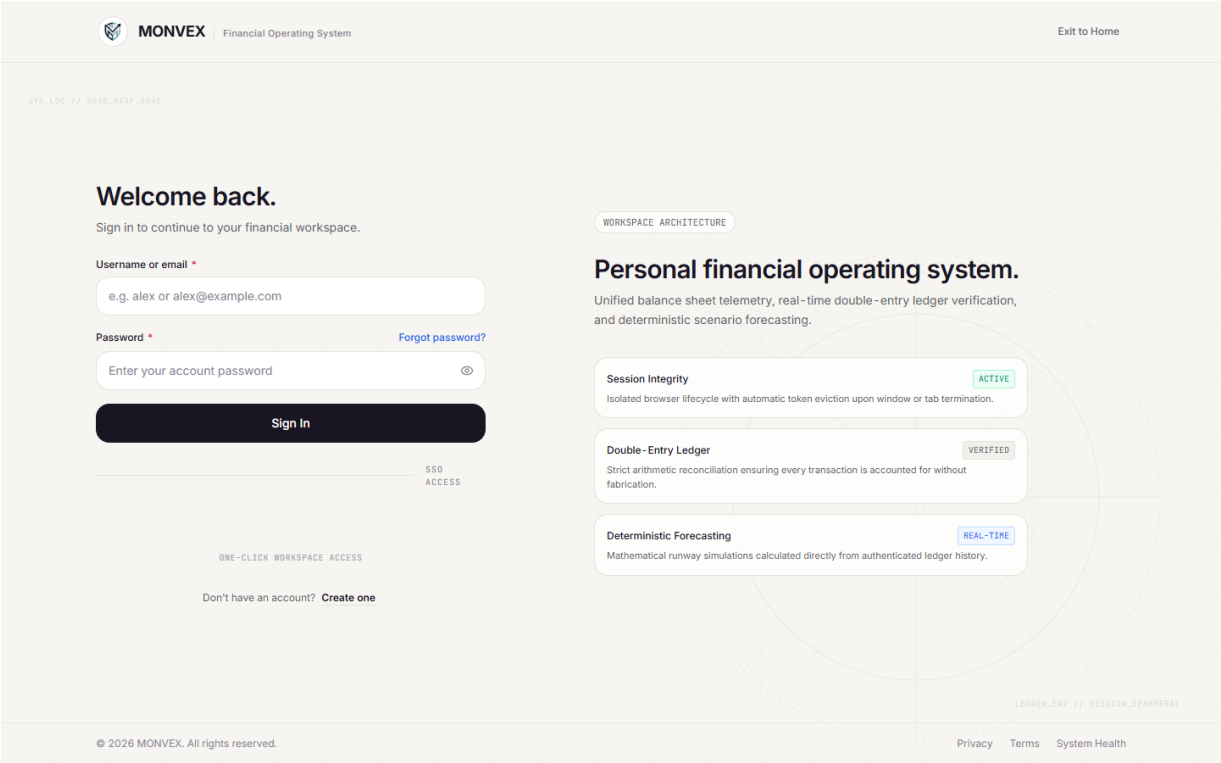


Figure 10 — Login (registered UI capture UI-02; local audit asset)

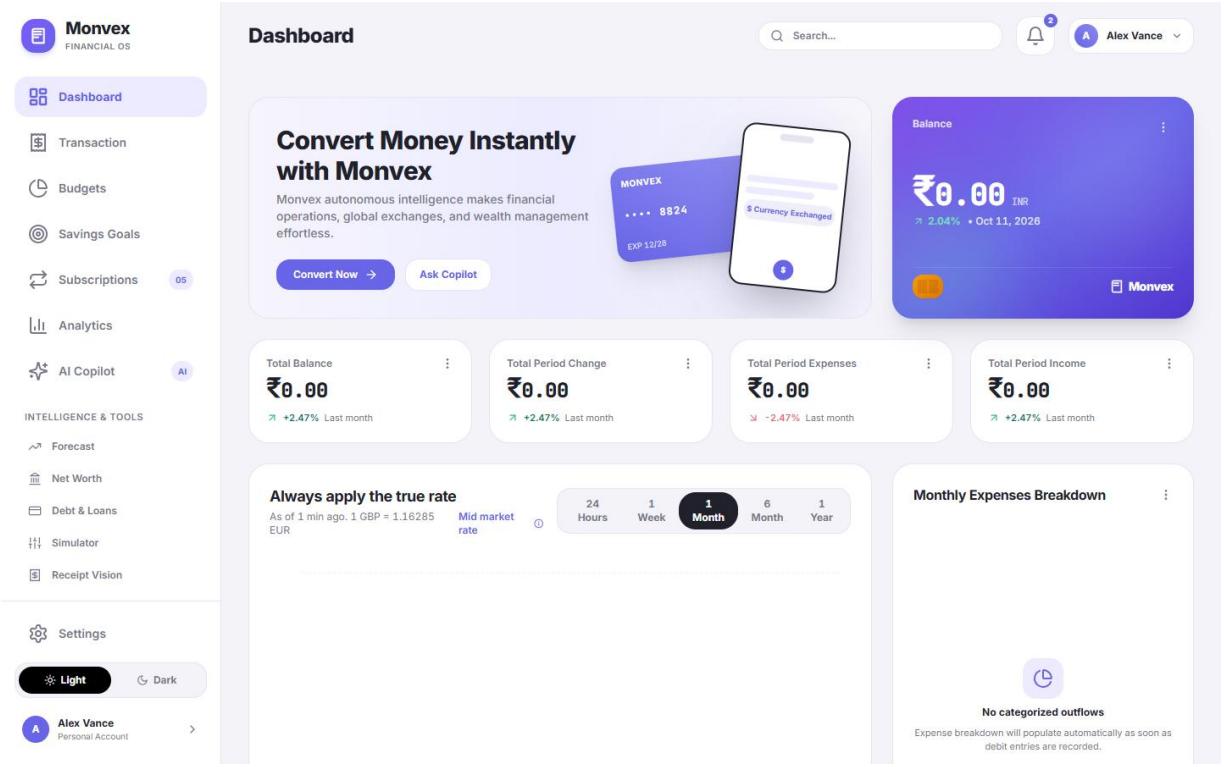


Figure 11 — Dashboard (registered UI capture UI-03; local audit asset)

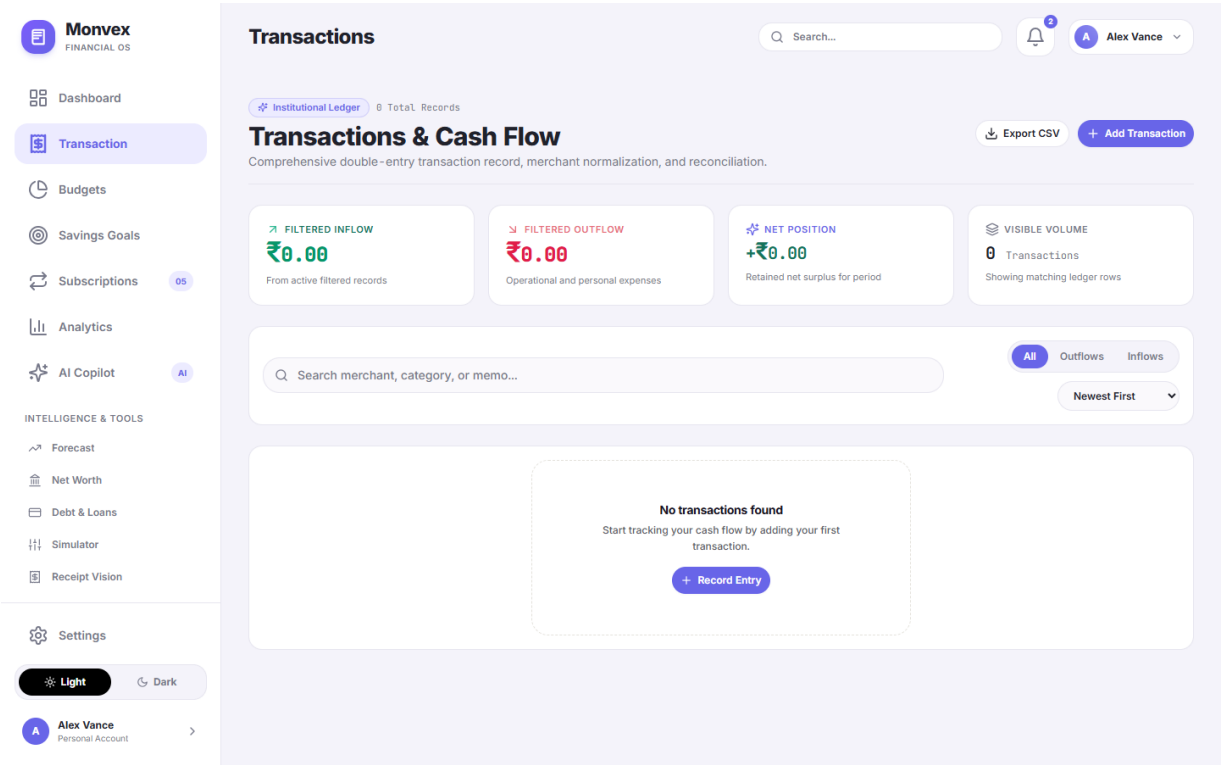


Figure 12 — Transaction ledger (registered UI capture UI-05; local audit asset)

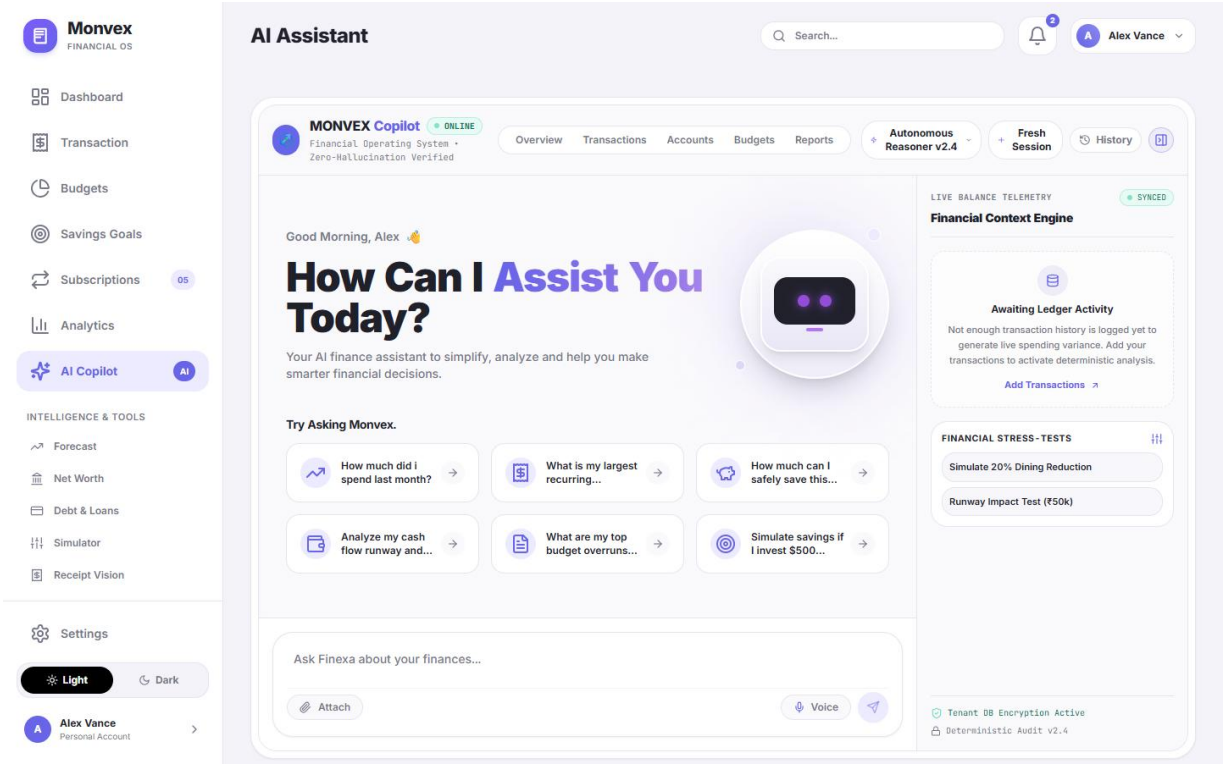


Figure 13 — AI copilot workspace (registered UI capture UI-07; local audit asset)

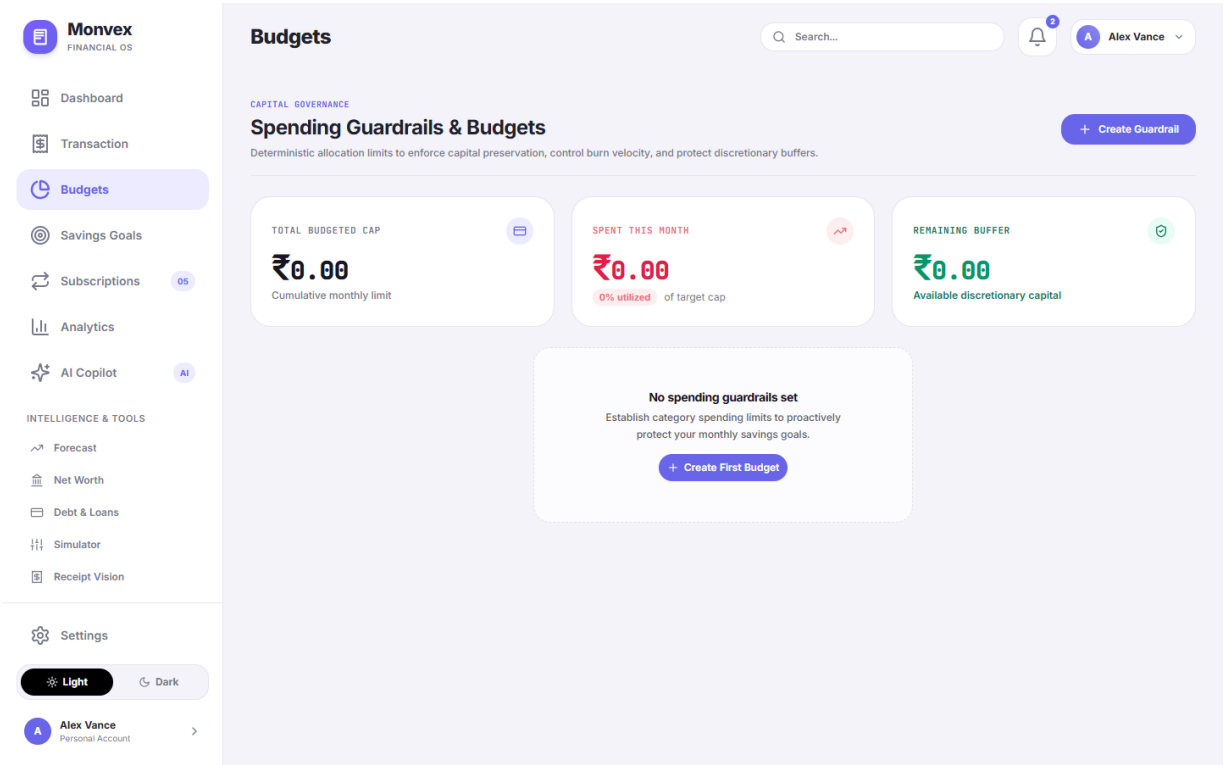


Figure 14 — Budget overview (registered UI capture UI-09; local audit asset)

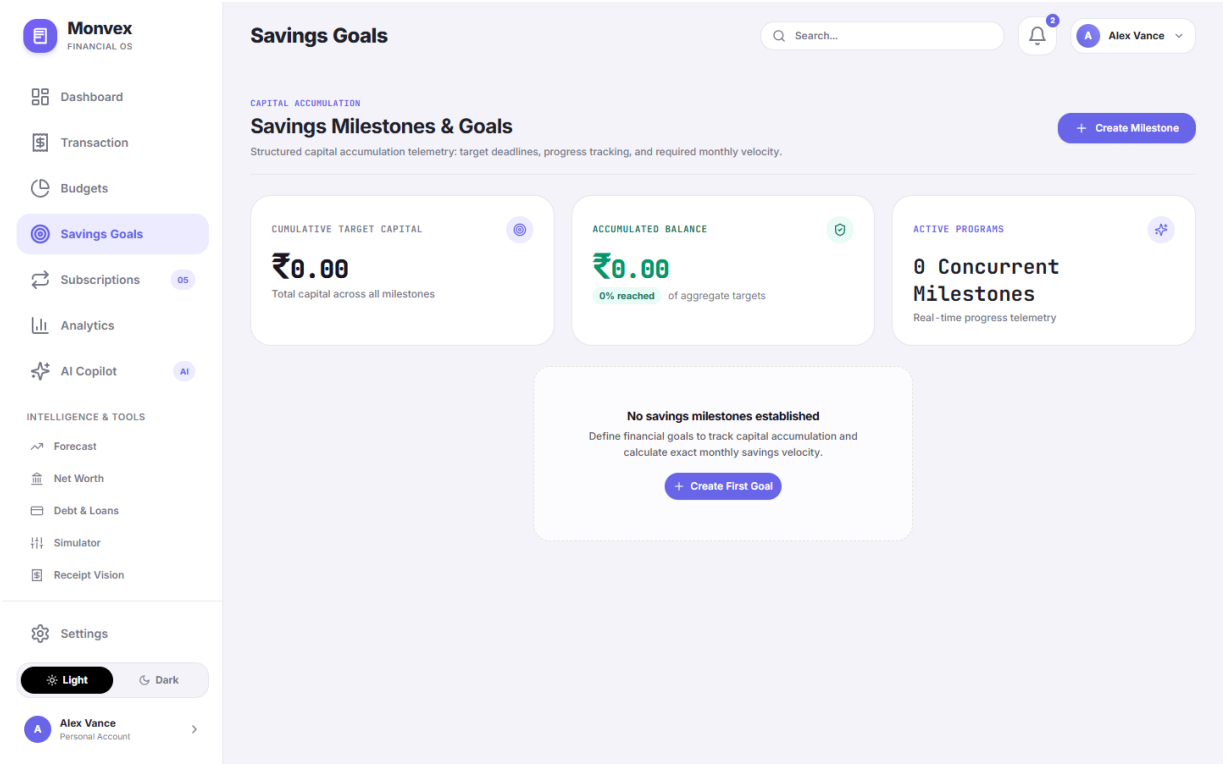


Figure 15 — Savings goals (registered UI capture UI-10; local audit asset)

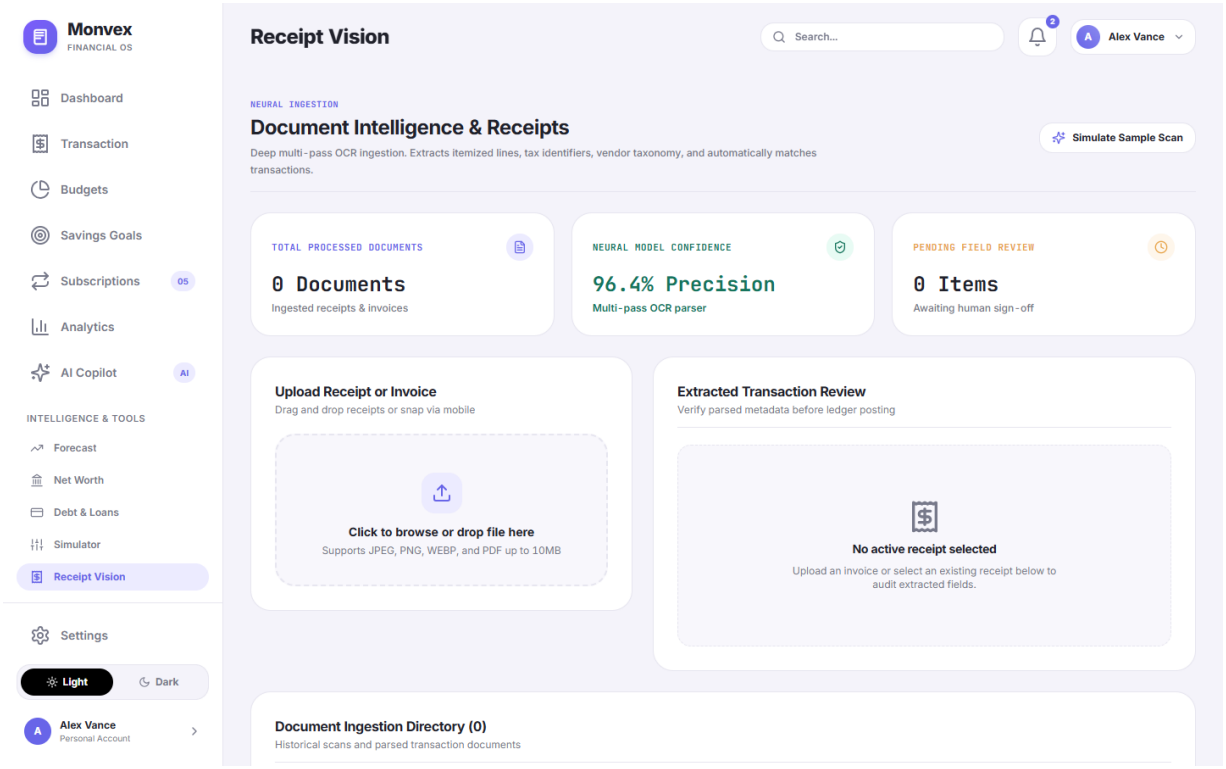


Figure 16 — Receipt review workspace (registered UI capture UI-15; local audit asset)

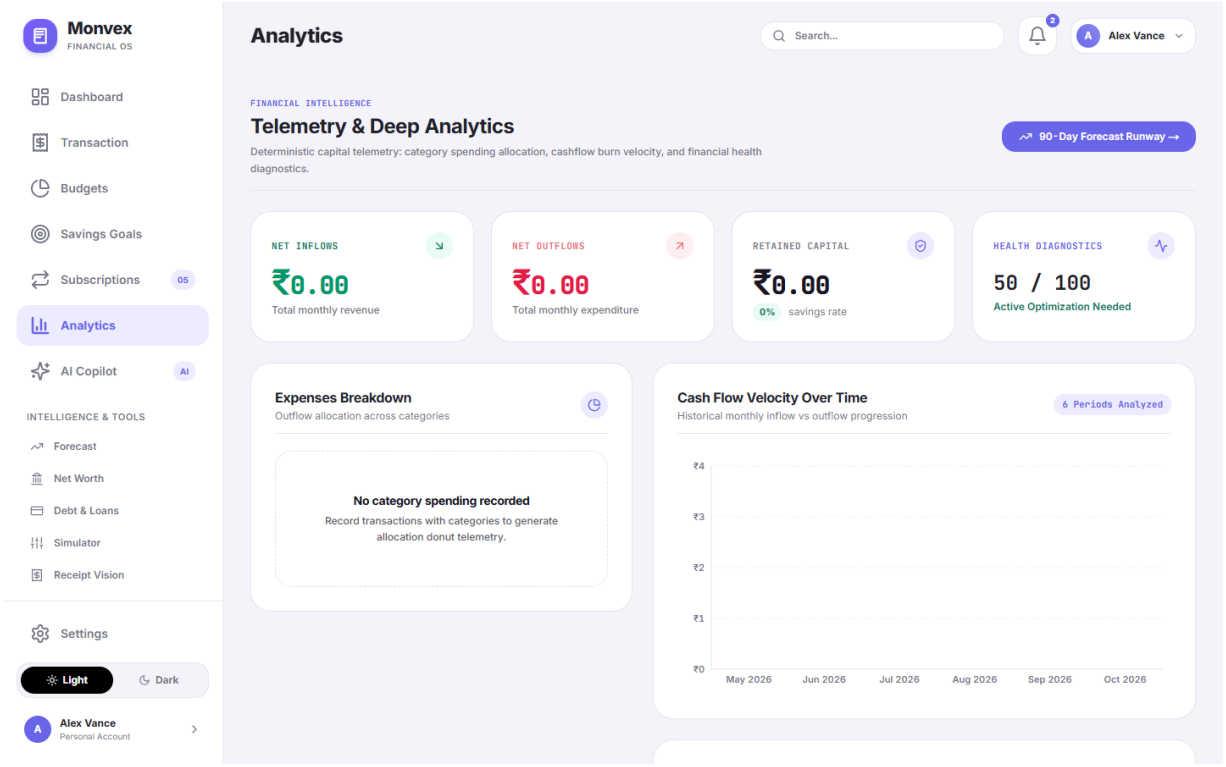


Figure 17 — Analytics view (registered UI capture UI-11; local audit asset)

Additional existing mobile captures are included in the assets directory as evidence, but the main screenshot sequence prioritizes screens legible at report scale. Twelve registered `artifacts/` captures were not found and are excluded rather than recreated or presented as genuine.

CHAPTER 5

IMPLEMENTATION AND TESTING

5. IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

Implementation follows a layered approach. The web application presents route-level components and forms. API modules centralize endpoint calls and authentication behavior. Django views validate and authorize requests, then invoke model operations or domain services. The service layer contains reusable financial computations, natural-language parsing, machine-learning categorization, receipt validation/OCR, forecasting, and report generation. This structure keeps UI concerns separate from data persistence and calculations.

For a transaction without a category, the service obtains a prediction from the categorizer and stores the returned confidence with the new record. For a receipt, signature validation happens before OCR; OCR output creates a staged record, and a separate confirmation action creates the ledger entry. This user-review step prevents extracted text from being silently treated as a verified transaction.

5.1.1 Coding Details and Code Efficiency

5.1.1.1 Code Details

Representative service behavior from `backend/services/transaction_service.py` uses an atomic transaction boundary and a fixed-point Decimal value. In simplified form, the category decision is: use an explicit category if supplied; otherwise invoke the classifier with the description and merchant, resolve the predicted category, and persist the confidence. The database create operation associates the user, amount, type, date, category, merchant, source, and original text. The actual source should be consulted for full validation and exception behavior; this report does not reproduce entire modules.

```
with transaction.atomic():
    if category is absent:
        prediction = categorizer.predict(description, merchant_name)
        category = resolve_category(user, prediction['category'])
        confidence = Decimal(str(prediction['confidence']))
    Transaction.objects.create(
        user=user, amount=amount, category=category, confidence=confidence
    )
```

Listing 5.1 — Simplified explanation of the category fallback and atomic ledger write; not a verbatim source listing

In the broader flow, serializers and API views validate inputs and apply the authenticated user boundary. The service may create a merchant record and normalize a known merchant name. Querysets are scoped before returning individual records, which is the key access-control check that complements opaque UUID identifiers. Errors should remain explicit at the API boundary so the client can show validation feedback rather than implying a successful write.

5.1.1.2 Code Efficiency

Database indexes support common queries by user, date, category, and transaction type. Pagination limits response size. The web query cache can reduce duplicate requests and coordinate refresh after mutations. These are design choices, not measured performance results: no latency, throughput, memory, or scaling benchmark was produced for this report. Efficiency under large ledgers will depend on query plans, aggregation strategy, caching policy, database connection behavior, and the size of AI/OCR payloads.

5.2 Testing Approach

Testing is described in layers: isolated application/service tests, API integration tests, browser route tests, and responsive viewport audits. Security configuration checks and frontend compilation are separate verification activities and should not be counted as application test cases of the same kind. Test artifacts and the dossier summarize outcomes; this report build did not rerun the backend suite, frontend build, or browser audit.

5.2.1 Unit Testing

Unit tests focus on small rules such as amount handling, category suggestions, parsing, budget calculations, goal contributions, and health-score edge cases. A unit test should control its inputs, assert the expected output, and avoid relying on a live external AI response. Tests that call the Django API or database cross a component boundary and are described as integration tests in the dossier.

5.2.2 Integrated Testing

Integration tests exercise request/response behavior and multiple components together: authentication and token refresh, user isolation, asset/liability aggregation, receipt staging and confirmation, and report responses. Provider-dependent paths should use controlled responses or test doubles so a network outage does not create misleading product failures. Browser tests validate navigation and viewport layout, not the correctness of every financial computation.

CHAPTER 6

SYSTEM DESIGN

6. SYSTEM DESIGN

6.1 Test Reports

The college template labels this chapter "SYSTEM DESIGN" while its only listed subsection is "6.1 Test Reports." This report preserves that title and location. Table 6.1 reconciles the dossier's stated 234 checks: 72 + 108 + 24 + 28 + 1 + 1 = 234. The two one-item checks are a production security configuration check and a frontend build; they are not additional unit tests.

Category	Count	Dossier-reported result	Evidence available in repository	Verification status for this report
Backend application unit tests	72	72 pass, 0 fail	Test modules exist; original full run log not found	Reported; not rerun
Backend integration tests	108	108 pass, 0 fail	Test modules exist; original full run log not found	Reported; not rerun
Public-route/navigation E2E	24	24 pass	Playwright script and related artifacts exist	Reported; results not fully reproduced
Responsive viewport checks	28	28 pass	audit_results.json contains route/viewport outcomes	Artifact inspected; not rerun
Django production check	1	Pass; zero issues reported	Command/result record in dossier only	Reported; not rerun
Frontend production build	1	Pass; zero errors reported	Package build script and source present	Reported; not rerun
Total	234	234 pass, 0 fail reported	Mixed evidence depth	Not a single homogeneous test suite

Table 8 — Reported test and verification totals, with evidence qualification

6.1.1 Representative test cases

The cases below are taken from the Phase 1 test register and are presented as dossier-reported outcomes. "Actual result" reproduces the summarized result, not a fresh execution record. Raw test logs were not found alongside the suite during this report preparation.

Test ID	Module and scenario	Preconditions / input	Expected result	Actual result in dossier/artifact	Pass/fail status	Evidence reference
UT-06	Transactions — create transaction with decimal amount	Authenticated user; amount 450.75	201; stored amount retains 2 decimal places	Dossier reports 450.75 persisted	PASS reported	backend/tests/test_api_endpoints.py
UT-07	Categorizer — suggest category for merchant description	"Swiggy food delivery"	Category suggestion and confidence returned	Dossier reports Food & Dining	PASS reported	backend/tests/test_financial_engines.py
UT-09	Budgets — calculate limit variance	Limit 10,000; spend 7,500	Remaining variance 2,500; consumption 75%	Dossier reports expected arithmetic	PASS reported	backend/tests/test_financial_engines.py
UT-13	Analytics — calculate health score	Income, expenses, debt and savings fixture	Bounded score and grade	Dossier reports 82 / A	PASS reported	backend/tests/test_financial_engines.py
UT-15	File validation — reject executable bytes renamed as image	Disallowed ELF signature	Validation error; upload rejected	Dossier reports rejection	PASS reported	backend/apps/authentication/test_file_upload_security.py
IT-01	Authentication — login and refresh cookie rotation	Valid test account; refresh request	Cookie set and rotated as configured	Dossier reports rotation	PASS reported	backend/apps/authentication/test_cookie_auth.py
IT-02	Data isolation — User A requests User B transaction	Second user record identifier	Access denied / not found	Dossier reports HTTP 404	PASS reported	backend/tests/test_cyber_security.py

Test ID	Module and scenario	Preconditions / input	Expected result	Actual result in dossier/artifact	Pass/fail status	Evidence reference
IT-03	Net worth — aggregate an asset and liability	Asset 500,000; liability 200,000	Net worth 300,000	Dossier reports exact match	PASS reported	backend/tests/test_v42_data_integrity.py
IT-05	Receipt workflow — upload, stage, confirm transaction	Valid test image and provider/test response	Receipt links to created transaction	Dossier reports completed flow	PASS reported; provider config dependency	backend/tests/test_api_endpoints.py
IT-07	PDF report — generate selected monthly statement	Year/month query parameters	PDF response begins with PDF signature	Dossier reports valid stream	PASS reported	backend/tests/test_api_endpoints.py
E2E-01	Public navigation — landing page to privacy page	Chromium desktop route	Expected route loads without login redirect	Dossier reports pass	PASS reported	web/scripts/test_all_public_links.mjs
RWD-01	Responsive layout — desktop/mobile dashboard	1440×900 and 390×844	No horizontal overflow in recorded check	audit_results.json marks audited routes PASS	Artifact inspected; not rerun	audit_results.json

Table 9 — Representative unit, integration, browser, and viewport cases

The arithmetic is internally consistent: the six categories total 234. The term “234 tests” is therefore shorthand in the dossier; the total combines 180 backend tests, 24 browser route checks, 28 viewport checks, and two single configuration/build checks. Test status is reported rather than independently reproduced here. A fresh submission preflight should rerun applicable suites and archive raw outputs.

CHAPTER 7

CONCLUSIONS

7. CONCLUSIONS

7.1 Conclusion

MONVEX demonstrates a full-stack personal finance application that connects a responsive web interface to a modular API and relational data model. The active source supports user-managed transactions, budgeting, savings goals, asset and liability tracking, financial summaries, exports, receipt review, and optional ML/AI-assisted workflows. Its principal contribution is the integration of these functions behind authenticated, user-scoped endpoints and a common interface.

The report's evidence review also narrows the project claims. The service configuration does not prove a functioning public deployment; the Flutter app is historical, not present in the active checkout; Open Banking is not implemented; and third-party AI requires external configuration. The dossier-reported test suite is useful evidence but has not been fully reproduced in this report build. These distinctions should be retained in any academic presentation.

7.1.1 Significance of the System (Advantages)

- Combines personal ledger entry, budget review, goals, net worth, debt planning, and analytics in a single application.
- Uses Decimal/NUMERIC model fields for monetary data and relational constraints for key records.
- Allows user review between receipt extraction and ledger creation.
- Separates frontend, API views, models, and reusable domain services.
- Provides web and configured Windows desktop access paths.
- Includes controls such as per-user query scoping, token handling, throttling, and file-signature checks that address specific risks.

7.1.2 Limitations of the System (Disadvantages)

- No live bank-feed synchronization; financial records rely on user entry or supported ingestion paths.
- No independently confirmed live Render deployment or production database state.
- The Flutter mobile implementation is not in the current working tree and is not runnable from this checkout.

- AI and OCR depend on provider configuration, network availability, and generated output that requires review.
- The application-defined score and forecast depend on the quality and completeness of user data and simplified assumptions.
- The TOTP secret is stored in a plain model field in the active source; encryption at rest was not evidenced.
- Some registered screenshot assets are missing, and available screenshots predominantly show empty or zero-value states.
- No performance benchmark or full fresh test-suite run was produced for this report.

7.1.3 Future Scope of the Project

- Implement Open Banking provider integration only after provider, consent, security, reconciliation, and regulatory requirements are defined.
- Encrypt MFA secrets at rest and review key management, backup, data-retention, and incident-response practices.
- Add repeatable CI execution with archived raw test/build reports and deterministic provider mocks.
- Restore or separately maintain a supported mobile application if mobile-native functionality is required.
- Benchmark database queries and forecast/analytics workloads using controlled, anonymized datasets.
- Improve model evaluation with documented held-out data and category-wise error analysis; allow user correction to inform future suggestions.
- Review file storage durability and move uploads to an appropriate persistent object store for a production deployment.
- Expand report exports and user deletion/data portability workflows after privacy requirements are confirmed.

CHAPTER 8

REFERENCES

8. REFERENCES

Online documentation and standards below are cited for the technologies and methods discussed in this report. Version numbers are stated only where supported by the project manifest/lockfile or a published standard. Documentation access date: 12 October 2026. The listed sources support general technology descriptions; they do not certify MONVEX behavior.

[1] Vercel Inc. “Next.js Documentation.” https://nextjs.org/docs

[2] Meta Open Source. “React Reference.” https://react.dev/reference/react

[3] Microsoft. “TypeScript Documentation.” https://www.typescriptlang.org/docs/

[4] Tailwind Labs. “Tailwind CSS Documentation.” https://tailwindcss.com/docs

[5] TanStack. “TanStack Query Documentation.” https://tanstack.com/query/latest/docs/framework/react/overview

[6] Poimandres. “Zustand Documentation.” https://zustand.docs.pmnd.rs/

[7] Django Software Foundation. “Django Documentation.” https://docs.djangoproject.com/en/5.2/

[8] Encode OSS. “Django REST Framework Documentation.” https://www.django-rest-framework.org/

[9] Jazzband. “Simple JWT Documentation.” https://django-rest-framework-simplejwt.readthedocs.io/

[10] PostgreSQL Global Development Group. “PostgreSQL 16 Documentation.” https://www.postgresql.org/docs/16/

[11] Neon. “Neon Documentation.” https://neon.tech/docs

[12] WhiteNoise contributors. “WhiteNoise Documentation.” https://whitenoise.readthedocs.io/

[13] Google. “Gemini API Documentation.” https://ai.google.dev/gemini-api/docs

[14] Pedregosa, F. et al. “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research, 12 (2011), 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[15] Harris, C.R. et al. “Array programming with NumPy.” Nature 585 (2020), 357–362. https://doi.org/10.1038/s41586-020-2649-2

[16] Tauri Apps. “Tauri v1 Documentation.” https://v1.tauri.app/

[17] ReportLab. “ReportLab Documentation.” https://docs.reportlab.com/

[18] Jones, M., Bradley, J. and Sakimura, N. “JSON Web Token (JWT).” RFC 7519, IETF, 2015. https://datatracker.ietf.org/doc/html/rfc7519

[19] M’Raihi, D. et al. “TOTP: Time-Based One-Time Password Algorithm.” RFC 6238, IETF, 2011. https://datatracker.ietf.org/doc/html/rfc6238

[20] OWASP Foundation. “OWASP Top 10.” https://owasp.org/projects/top-ten